

Day10

Karnish N

(18/08/2025)

Task 1

1. Create Fast API, to store, retrieve all and retrieve specific song by Id.
SQLite can be used as data store
table can contain
song name, playing time in seconds, album (or movie) along with Id

[Song.py](#)

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

from fastapi import FastAPI, Depends, HTTPException
import uvicorn
from pydantic import BaseModel
from typing import List

SQLALCHEMY_DATABASE_URL = "sqlite:///D:/New folder/SongDB.db"
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    echo=True,
    connect_args={"check_same_thread": False}
)

Base = declarative_base()

class Song(Base):
    __tablename__ = "Song"
    Id = Column(Integer, primary_key=True, index=True,
autoincrement=True)
    Name = Column(String, nullable=False)
    Duration = Column(Integer, nullable=False)

Base.metadata.create_all(bind=engine)

SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
```

```

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class SongCreate(BaseModel):
    Name: str
    Duration: int
    class Config:
        from_attributes = True

class SongResponse(BaseModel):
    Id: int
    Name: str
    Duration: int
    class Config:
        from_attributes = True

@app.post("/add_song", response_model=SongResponse)
def add_song(song: SongCreate, db: Session = Depends(get_db)):
    s = Song(Name=song.Name, Duration=song.Duration)
    db.add(s)
    db.commit()
    db.refresh(s)
    return s

@app.get('/songs', response_model=List[SongResponse])
def get_all(db: Session = Depends(get_db)):
    recs = db.query(Song).all()
    return recs

@app.get("/song/{song_id}", response_model=SongResponse)
def get_by_id(song_id: int, db: Session = Depends(get_db)):
    s = db.query(Song).filter_by(Id=song_id).first()
    if not s:
        raise HTTPException(status_code=404, detail="Song not found")
    return s

```

```
if __name__ == "__main__":
    uvicorn.run("data:app", host="127.0.0.1", port=8000, reload=True)
```

OUTPUT:

```
(Background on this error at: https://sqlalche.me/e/20/e3q8)
(my_fast) PS D:\visual-studio\pythonNew\Database> python data.py
D:\visual-studio\pythonNew\Database\data.py:17: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (
eprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
  Base = declarative_base()
2025-08-18 16:46:15,629 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-08-18 16:46:15,630 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("Song")
2025-08-18 16:46:15,630 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-08-18 16:46:15,636 INFO sqlalchemy.engine.Engine COMMIT
INFO: Will watch for changes in these directories: ['D:\\visual-studio\\pythonNew\\Database']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [5476] using StatReload
2025-08-18 16:46:17,385 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-08-18 16:46:17,386 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("Song")
2025-08-18 16:46:17,387 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-08-18 16:46:17,388 INFO sqlalchemy.engine.Engine COMMIT
2025-08-18 16:46:17,424 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-08-18 16:46:17,426 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("Song")
2025-08-18 16:46:17,428 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-08-18 16:46:17,429 INFO sqlalchemy.engine.Engine COMMIT
INFO: Started reloader process [5476]
```

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

POST	/add_song	Add Song	✓
GET	/songs	Get All	▼
GET	/song/{song_id}	Get By Id	▼

Schemas

HTTPValidationError > Expand all object

SongCreate > Expand all object

SongResponse > Expand all object

ValidationError > Expand all object

Code	Description	Links
200	Successful Response	No links
Media type		
application/json		
Controls Accept header.		
Example Value Schema		
<pre>{ "Id": 0, "Name": "string", "Duration": 0 }</pre>		
422	Validation Error	No links
Media type		
application/json		
Example Value Schema		
<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>		

Curl	
<pre>curl -X 'GET' \ 'http://127.0.0.1:8000/songs' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/songs</pre>	
Server response	
Code	Details
200	Response body
<pre>[{ "Id": 1, "Name": "Shape of You", "Duration": 240 }, { "Id": 2, "Name": "Believer", "Duration": 210 }, { "Id": 3, "Name": "Blinding Lights", "Duration": 200 }, { "Id": 4, "Name": "Peaches", "Duration": 215 }, { "Id": 5, "Name": "Stay", "Duration": 141 }, { "Id": 6, "Name": "Bad Guy" }]</pre>	
Download	

Name	Description
song_id * required integer (path)	1
Execute Clear	
Responses	
Curl	
<pre>curl -X 'GET' \ 'http://127.0.0.1:8000/song/1' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/song/1</pre>	
Server response	
Code	Details
200	Response body
	<pre>{ "Id": 1, "Name": "Shape of You", "Duration": 240 }</pre>



SQL 1*

SELECT * FROM Song

Id	Name	Duration
1	Shape of You	240
2	Believer	210
3	Blinding Lights	200
4	Peaches	215
5	Stay	141
6	Bad Guy	194